

Estándar Protocolo CTF - CC8 2026

Ficha Técnica de Especificación	
Campo	Detalle
Versión del documento	1.2.0
Versión del protocolo (v)	1
Estado	Vigente (Acuerdo de clase)
Última modificación	2026-07-25
Historial de cambios	Ver sección 7. Control de cambios al final del documento.

ABSTRACT

Este documento define el protocolo estándar para la competencia Capture The Flag (CTF) de la clase CC8 2026. Establece las bases técnicas de comunicación, el formato de mensajes JSON y las reglas de juego para garantizar la interoperabilidad entre diversos proyectos de software.

TERMINOLOGÍA

Las palabras clave de esta especificación tienen carácter normativo:

- **DEBE / NO DEBE:** Requisitos absolutos para la compatibilidad. Todo proyecto que no cumpla un **DEBE** es incompatible con el resto de la clase.
- **RECOMENDADO:** Prácticas sugeridas para estabilidad.
- **Coalescencia:** Capacidad del servidor de omitir estados antiguos en favor del más reciente ante clientes lentos.

TABLA DE CONTENIDOS

ABSTRACT.....	1
TERMINOLOGÍA.....	1
1. CONEXIÓN.....	4
1.1 Transporte.....	4
1.2 Puertos.....	4
1.3 Descubrimiento de servidores.....	4
2. EL IDIOMA DE LOS MENSAJES.....	6
2.1 Delimitación de mensajes (framing).....	6
2.2 Formato y codificación.....	6
2.3 Catálogo de mensajes.....	7
2.3.1 <i>Vista general del catálogo</i>	7
2.3.2 <i>Detalle de campos por mensaje</i>	8
2.3.3 <i>Constantes y límites acordados</i>	10
3. REGLAS DE LA PARTIDA.....	12
3.1 Secuencia de la partida.....	12
3.2 Sistema de coordenadas.....	12
3.3 Reglas del dominio (normativas).....	12
4. AUTORIDAD Y SINCRONIZACIÓN.....	14
4.1 Autoridad y validaciones.....	14
4.2 Sincronización del estado.....	15
5. MANEJO DE FALLAS Y DESCONEXIONES.....	16
5.1 Manejo de errores.....	16
5.1.1 <i>Tabla de errores comunes</i>	16
5.2 Desconexiones.....	17
5.2.1 <i>Casos especiales de desconexión</i>	17
5.3 Decisiones de empate y propiedades del dominio.....	18
5.3.1 <i>Propiedades que deben permanecer verdaderas</i>	18
6. CONSIDERACIONES DE SEGURIDAD.....	20
6.1 Ausencia de Cifrado y Autenticación.....	20
6.2 Denegación de Servicio (DoS) y Control de Memoria.....	20
6.3 Validación de Entradas e Inyección en el Parser JSON.....	20
6.4 Integridad de la Partida y Prevención de Trampas.....	20
7. CONTROL DE CAMBIOS.....	22
7.1 Versionado.....	22
7.2 Procedimiento para modificar el estándar.....	22
7.3 Historial de cambios.....	22

1. CONEXIÓN

Diagrama de Conexión:

```
Cliente (UDP:8888) --discover--> Red Local (Broadcast)
Servidor (UDP:Auto) <--server_info-- Cliente (Unicast)
Cliente (TCP:tcp_port) --join--> Servidor
```

1.1 Transporte

Define la tecnología encargada de mover los bytes entre las máquinas participantes y las garantías de entrega que esta ofrece. Todos los proyectos de la clase deben acordar el mismo transporte para que sus conexiones sean compatibles entre sí. Se establece un esquema híbrido:

- **TCP:** Para toda la comunicación de la partida (unirse, moverse, tomar la bandera, estado del juego, fin de partida). Garantiza que los mensajes lleguen completos y en el orden en que se enviaron.
- **UDP:** Exclusivamente para el descubrimiento de servidores en la red local (desarrollado en la sección 1.3).

Requisitos para cada proyecto:

- Usar los sockets básicos que ya trae el lenguaje, sin librerías externas de conexión (por ejemplo: socket en Python, net en Go, TcpClient en C#, net en Node —nunca ws—, POSIX/Winsock en C++, StreamPeerTCP en Godot, dart:io en Dart).
- En modo cliente, abrir una única conexión TCP hacia el servidor y utilizarla para todo el intercambio de mensajes del juego.
- En modo servidor, aceptar y mantener conexiones TCP simultáneas de hasta 100 clientes.
- Mantener, además, un socket UDP independiente dedicado solo al descubrimiento.

1.2 Puertos

Establece los números de puerto en los que los servidores escuchan, tanto para el descubrimiento como para la partida. Separar ambos puertos evita que la búsqueda de servidores interfiera con el tráfico del juego en curso. Se definen dos tipos de puerto:

- **Puerto de descubrimiento (UDP):** Fijo e igual para todos los proyectos de la clase: **8888**. Todo servidor debe escuchar ahí sin excepción.
- **Puerto de juego (TCP):** Cada servidor lo elige libremente (por ejemplo, 8889) y no necesita coincidir entre proyectos.

Requisitos para cada proyecto:

- El servidor debe anunciar su puerto TCP dentro del mensaje de respuesta al descubrimiento, ya que el cliente no lo conoce de antemano.
- El cliente primero pregunta por el puerto UDP fijo (8888), lee el puerto TCP que el servidor le indica, y con ese puerto abre la conexión de juego.
- Ningún proyecto debe hardcodear un puerto TCP ajeno: siempre se obtiene dinámicamente de la respuesta de descubrimiento (o del respaldo manual **IP:puerto**, sección 1.3).

1.3 Descubrimiento de servidores

Describe el procedimiento mediante el cual un cliente localiza los servidores disponibles en la red local sin conocer sus direcciones IP de antemano. Este mecanismo depende del puerto UDP definido en la sección 1.2. El descubrimiento sigue dos vías:

- **Automática (broadcast UDP):** El cliente envía una pregunta a toda la red y cada servidor activo responde de forma individual.
- El cliente no conoce la IP del servidor, por lo que envía el paquete de solicitud a **ambas** direcciones de broadcast: la limitada (**255.255.255.255:8888**) y la de su subred (por ejemplo, **192.168.1.255:8888**, calculada a partir de su propia IP y máscara). Muchos routers no reenvían 255.255.255.255; enviar a ambas maximiza la probabilidad de descubrimiento.
- El socket UDP del cliente DEBE habilitar la opción **SO_BROADCAST** antes de enviar.
 - Este paquete se envía a todos los dispositivos conectados a esa red local.
 - El servidor examina la IP de origen del paquete de solicitud (la IP del cliente) para saber a dónde responder.
 - El servidor envía un paquete de respuesta directamente a la IP del cliente.
 - Al recibir la respuesta, el cliente comprueba la IP de origen del paquete (que es la IP real del servidor en la LAN) y lo muestra en la lista.
- **Manual (respaldo):** El cliente permite escribir directamente la IP de un servidor, para los casos en que el broadcast no funcione (router o WiFi que lo bloquee). En este modo el cliente envía el mismo mensaje **discover** por **unicast** a **IP:8888** (el broadcast puede estar bloqueado, pero el unicast UDP funciona). Como respaldo adicional, el cliente DEBE aceptar también el formato **IP:puerto** para conectarse directamente por TCP sin pasar por el descubrimiento.

Requisitos para cada proyecto:

- El cliente debe enviar por broadcast —a 255.255.255.255:8888 y al broadcast de su subred— un mensaje del tipo:
`{"type": "discover", "v": 1}`
- El servidor debe responder directamente al remitente con:
`{"type": "server_info", "v": 1, "name": "...", "tcp_port": 8889, "state": "lobby", "players": 3}`
- El campo **state** de **server_info** solo admite dos valores: **"lobby"** si el servidor acepta nuevos jugadores en este momento, y **"playing"** en cualquier otro caso (countdown, partida en curso o pausa de resultados).
- Un **discover** con **v** distinto de 1 se descarta silenciosamente (no se responde). Un datagrama UDP que no sea JSON válido también se descarta silenciosamente (no existe conexión donde devolver un **error**).
- El servidor DEBE abrir su socket UDP de descubrimiento con **SO_REUSEADDR** (y **SO_REUSEPORT** donde la plataforma lo permita), para tolerar reinicios rápidos y pruebas locales con más de un proceso en la misma máquina.
- El cliente debe mostrar la lista de servidores encontrados y permitir al usuario elegir uno.
- El cliente debe incluir, sin excepción, la opción de conexión manual por IP como respaldo al broadcast (ver vía Manual arriba).

2. EL IDIOMA DE LOS MENSAJES

Diagrama de Framing:

```
[JSON Message 1]\n[JSON Message 2]\n[Part of JSON 3...]
```

2.1 Delimitación de mensajes (framing)

Especifica la regla que permite identificar dónde termina un mensaje y dónde comienza el siguiente. Este punto es indispensable porque TCP entrega los datos como un chorro continuo de bytes, sin ninguna separación natural entre mensajes: dos mensajes pueden llegar pegados en una sola lectura, o uno solo puede llegar partido en dos lecturas distintas. Se define la siguiente regla:

- **Un mensaje = una línea:** Cada mensaje es un texto JSON completo, seguido del carácter de salto de línea (`\n`).
- El JSON de un mensaje no puede contener saltos de línea internos: debe escribirse siempre "aplastado" en una sola línea (sin indentación ni formato bonito).
- El salto de línea (`\n`) es exclusivamente el separador entre mensajes; nunca debe aparecer dentro del contenido de un campo.

Procedimiento obligatorio para leer mensajes:

1. Ir acumulando los bytes que llegan por el socket en un buffer (un cajón temporal).
2. Cada vez que llegue un `\n` dentro de ese buffer, cortar ahí: todo lo acumulado ANTES del `\n` es un mensaje completo.
3. Convertir ese texto cortado en un objeto JSON (parsear).
4. Lo que quede DESPUÉS del `\n` se conserva en el buffer, porque puede ser el inicio del siguiente mensaje (o estar incompleto todavía).
5. Repetir mientras la conexión siga abierta.

Aclaración importante: Esta regla aplica únicamente a mensajes enviados por TCP. Los mensajes de descubrimiento, enviados por UDP, se transmiten como paquetes completos en un solo paso y no requieren el carácter `\n` ni el proceso de buffer.

Reglas adicionales de framing:

- El emisor nunca incluye el carácter `\r` (retorno de carro) en sus mensajes. El receptor PUEDE tolerar un `\r` inmediatamente antes del `\n` (fines de línea Windows) y DEBE descartarlo antes de parsear.
Ningún mensaje puede superar `message_max_size` (64 KB) incluyendo el `\n` final; el límite aplica también a datagramas UDP. Un mensaje TCP que lo supere se rechaza con `MESSAGE_TOO_LARGE` y la conexión se cierra; un datagrama UDP que lo supere se descarta silenciosamente.

2.2 Formato y codificación

Define el lenguaje en que se escriben los mensajes y la codificación de caracteres utilizada, de modo que todos los proyectos interpreten la misma información de forma idéntica, sin importar el lenguaje de programación en que estén escritos.

- **Formato:** JSON. Todo mensaje del protocolo debe ser un objeto JSON válido, no un texto libre ni un formato inventado por cada proyecto.
- **Codificación:** UTF-8. Todo el texto (nombres de jugadores, motivos de error, etc.) debe

codificarse y decodificarse en UTF-8 en ambos extremos de la conexión.

- **Campo identificador obligatorio:** Todo mensaje, sin excepción, debe incluir un campo llamado "type" como texto (string), cuyo valor identifica de qué mensaje se trata. Ningún mensaje puede omitirlo. Ejemplo: `{"type": "join", ...}`.
- **Valores numéricos:** Los campos de posición y dirección (x, y, dir) se representan como números, nunca como texto (ejemplo: "x": -1, no "x": "-1").
- **Estructura plana:** Los mensajes no deben anidar más de dos niveles de profundidad (por ejemplo, config dentro de welcome es aceptable; un tercer nivel dentro de config no lo es), para mantener el parseo simple en todos los lenguajes.
- **Versión:** El campo v, donde aplique (`discover`, `server_info`, `join`), DEBE ser exactamente el entero 1. Ningún otro mensaje lleva v.
- **Campos desconocidos:** El receptor DEBE ignorar silenciosamente cualquier campo no documentado en este estándar (lectura tolerante). Esto permite agregar campos opcionales en el futuro sin romper a los demás proyectos.
- **Orden de procesamiento (servidor):** El servidor procesa los mensajes entrantes de a uno, en el orden en que llegan por TCP, y aplica cada uno por completo antes de evaluar el siguiente. Este orden de llegada es el orden oficial de la partida y la única regla de desempate (ver sección 5.3).

2.3 Catálogo de mensajes

2.3.1 Vista general del catálogo

Tipo	Dirección	Fase	Propósito	Campos principales
discover	C → UDP	Cualquiera	Buscar servidores en la red	v
server_info	S → UDP	Cualquiera	Anunciar el servidor encontrado	v, name, tcp_port, state, players
join	C → S	Lobby	Unirse a la partida	v, name
input	C → S	Playing	Comunicar hacia dónde se mueve	dir (dir.x, dir.y)
interact	C → S	Playing	Tomar o robar la bandera	ninguno
welcome	S → C	Lobby	Asignar identidad y constantes	player_id, config
lobby	S → C	Lobby	Lista de jugadores en espera	players
countdown	S → C	Countdown	Mostrar la cuenta regresiva	seconds
start	S → C	Countdown-Playin	Iniciar la partida	ninguno

Tipo	Dirección	Fase	Propósito	Campos principales
		g		
state	S → C	Playing	Replicar el mundo del juego	flag, players
game_over	S → C	Playing-Finished	Anunciar al ganador	winner
error	S → C	Cualquiera	Rechazar una acción inválida	reason

2.3.2 Detalle de campos por mensaje

1. Mensajes de descubrimiento (UDP):

- **discover** (Cliente → Broadcast UDP | Fase: cualquiera):
 - **v** (entero): Versión del protocolo que habla el cliente.
- **server_info** (Servidor → Unicast UDP | Fase: cualquiera):
 - **v** (entero): Versión del protocolo que habla el servidor.
 - **name** (texto): Nombre del servidor.
 - **tcp_port** (entero): Puerto TCP donde escucha el juego.
 - **state** (texto): "lobby" si acepta nuevos jugadores en este momento; "playing" en cualquier otra fase (countdown, partida o pausa de resultados).
 - **players** (entero): Cantidad de jugadores conectados.

2. Mensajes de cliente a servidor (TCP):

- **join** (Cliente → Servidor | Fase: lobby):
 - **v** (entero): Versión del protocolo que habla el cliente. DEBE ser exactamente 1; de lo contrario el servidor responde **VERSION_MISMATCH** y cierra la conexión.
 - **name** (texto): Nombre del jugador. Reglas: después de recortar espacios al inicio y al final (trim), longitud entre 1 y **name_max_length** (20) caracteres UTF-8, sin caracteres de control ni saltos de línea. Los nombres **no** necesitan ser únicos: el **player_id** es quien distingue a los jugadores. Si el nombre es inválido, el servidor responde **NAME_INVALID** (sin cerrar) y el cliente puede reintentar con otro nombre.
 - Un segundo **join** en la misma conexión se rechaza con **INVALID_PHASE** (sin cerrar la conexión).
 - Un **join** recibido cuando el servidor está en **countdown** o **playing** se rechaza con **GAME_STARTED** y la conexión se cierra (ver tabla de errores, sección 5.1).
- **input** (Cliente → Servidor | Fase: playing):
 - **dir.x** (entero): -1 = izquierda, 0 = quieto, 1 = derecha.
 - **dir.y** (entero): -1 = arriba, 0 = quieto, 1 = abajo.
- **interact** (Cliente → Servidor | Fase: playing):
 - *Sin campos*: Intenta tomar la bandera libre o robarla al portador.

Combinaciones posibles de dir (x, y):

- (-1, -1): Arriba-Izquierda

- (0, -1): Arriba
- (1, -1): Arriba-Derecha
- (-1, 0): Izquierda
- (0, 0): Quieto
- (1, 0): Derecha
- (-1, 1): Abajo-Izquierda
- (0, 1): Abajo
- (1, 1): Abajo-Derecha

3. Mensajes de servidor a cliente (TCP):

- **welcome** (Servidor → Cliente | Fase: lobby):
 - **player_id** (texto): Identificador único que el servidor le asigna al jugador.
 - **config.map_size** (número): Lado del mapa en unidades lógicas.
 - **config.circle_radius** (número): Radio del círculo central.
 - **config.player_radius** (número): Radio del cuerpo del jugador.
 - **config.interact_radius** (número): Distancia máxima para tomar o robar la bandera.
 - **config.speed** (número): Velocidad de movimiento en unidades por segundo.
 - **config.tick_rate** (entero): Envíos de estado por segundo.
- **lobby** (Servidor → Cliente | Fase: lobby):
 - **players[].id** (texto): Identificador de cada jugador conectado.
 - **players[].name** (texto): Nombre visible de cada jugador conectado.
 - **Cuándo se envía lobby**: (a) a cada cliente inmediatamente después de su **welcome**; (b) en broadcast a todos los clientes del lobby ante cada alta o baja; (c) en broadcast como señal de **regreso al lobby** cuando se aborta el countdown (sección 5.2) o cuando termina la partida (sección 3.1). Recibir un **lobby** fuera de la fase lobby significa siempre "el servidor volvió a la fase lobby": el cliente abandona la pantalla actual y muestra la sala de espera.
- **countdown** (Servidor → Cliente | Fase: countdown):
 - **seconds** (entero): Segundos restantes para el inicio (5, 4, 3, 2, 1).
 - **Temporización del countdown**: El servidor envía exactamente un **countdown** por segundo, con **seconds** = 5, 4, 3, 2, 1, y envía **start** inmediatamente después del último. El cliente NO DEBE enviar **input** ni **interact** antes de recibir **start**; el servidor rechaza esos mensajes con **INVALID_PHASE**.
- **start** (Servidor → Cliente | Fase: countdown → playing):
 - *Sin campos*: Marca el inicio exacto de la partida.
- **state** (Servidor → Cliente | Fase: playing):
 - **flag.owner** (texto o nulo): ID del portador, o null si la bandera está libre. El valor de bandera libre es siempre **null** (nunca 0, nunca cadena vacía).
 - **flag.x, flag.y** (número): Posición actual de la bandera. Cuando **flag.owner** no es **null**, el servidor DEBE transmitir la posición del portador (**flag.x** = x del portador, **flag.y** = y del portador) en cada envío, para que todos los clientes dibujen la bandera pegada al portador.
 - **players[].id** (texto): Identificador del jugador.
 - **players[].x, players[].y** (número): Posición actual de ese jugador (1 decimal; redondeo half-away-from-zero; los clientes no deben comparar posiciones por igualdad exacta).
- **players[]** incluye **únicamente** a los jugadores conectados en ese instante, incluido el propio cliente que recibe el mensaje. Si un **id** presente en un **state** desaparece en el siguiente, el cliente elimina ese avatar de inmediato (no existe mensaje de baja; ver

sección 5.2).

- **state** es un mensaje **coalescible**: si un cliente se atrasa, el servidor puede descartar envíos pendientes y mandar solo el más reciente. Los demás mensajes (**lobby**, **countdown**, **start**, **game_over**, **error**) nunca se descartan.
- **game_over** (Servidor → Cliente | Fase: playing → finished):
 - **winner** (texto): ID del jugador que ganó la partida.
- **Después del game_over**: El servidor mantiene las conexiones TCP abiertas, espera **post_game_seconds** (5 segundos, para que los clientes muestren la pantalla de victoria), revierte su estado a **lobby** y envía en broadcast un mensaje **lobby** actualizado. Nadie necesita reconectarse para jugar otra partida (ver secuencia completa en sección 3.1).
- **error** (Servidor → Cliente | Fase: cualquiera):
 - **reason** (texto): Código del motivo del rechazo, siempre en MAYÚSCULAS_CON_GUIONES_BAJOS y tomado del catálogo de la sección 5.1 (ej. "GAME_STARTED", "LOBBY_FULL"). Los clientes deben poder mostrarlo al usuario tal cual.

2.3.3 Constantes y límites acordados

Categoría	Constante / Límite	Valor	Significado
Constantes en welcome.config	map_size	1000	El mapa mide 1000 x 1000 unidades lógicas.
	circle_radius	300	El círculo central mide 300 unidades de radio.
	player_radius	15	El cuerpo del jugador mide 15 unidades de radio.
	interact_radius	40	Hasta 40 unidades de distancia para tomar o robar la bandera.
	speed	200	200 unidades por segundo de velocidad de movimiento.
	tick_rate	20	20 envíos de estado por segundo.
Constantes del Servidor	countdown_seconds	5	La cuenta regresiva dura 5 segundos antes de iniciar.
	min_players	2	Mínimo de jugadores para disparar y mantener el countdown.

Categoría	Constante / Límite	Valor	Significado
	post_game_seconds	5	Pausa tras game_over antes del regreso al lobby.
	circle_center	(500, 500)	Centro del mapa y del círculo (= map_size / 2).
	spawn_radius_min / max	350 / 450	Anillo de aparición alrededor del centro (sección 3.3).
	victory_distance	315	circle_radius + player_radius; distancia al centro que se debe superar para ganar (sección 3.3).
Límites del Protocolo	max_players	100	Máximo de jugadores por partida (según el enunciado).
	name_max_length	20	Largo máximo del nombre del jugador en caracteres.
	message_max_size	64 KB	Tamaño máximo de un mensaje individual.

Los valores de welcome.config son constantes fijas, no configurables. Todo servidor de la clase DEBE anunciar exactamente los valores de la tabla (`map_size: 1000`, `circle_radius: 300`, `player_radius: 15`, `interact_radius: 40`, `speed: 200`, `tick_rate: 20`). Se envían dentro de `welcome` para que el cliente no los tenga que hardcodear y para que el mensaje sea autodescriptivo, pero ningún servidor puede cambiarlos: si cada servidor usara velocidades, radios o mapas distintos, el mismo jugador se movería más rápido o ganaría con reglas diferentes según a quién se conecte, y las partidas entre proyectos dejarían de ser justas y comparables. Se mantienen en el mensaje como documentación viva del contrato, no como parámetros dinámicos.

3. REGLAS DE LA PARTIDA

Ciclo de Vida:

Lobby -> Countdown (5s) -> Playing -> Game Over -> Pausa (5s) -> Lobby

3.1 Secuencia de la partida

1. Búsqueda de servidores disponibles (descubrimiento).
2. Lobby / espera al comienzo de la partida.
3. Countdown.
4. Inicio de la partida:
 - Ubicación inicial de los jugadores (spawn aleatorio, fuera del círculo).
 - Movimiento libre de los jugadores.
5. Eventos durante la partida (ocurren en cualquier momento y cualquier cantidad de veces):
 - Captura de bandera.
 - Robo de bandera.
 - Salida del círculo con la bandera (condición de victoria).
6. Fin de la partida / mostrar ganador (`game_over`).
7. Transición post-partida: el servidor espera `post_game_seconds` (5 s) con las conexiones TCP abiertas, revierte su estado a lobby y hace broadcast de un mensaje lobby actualizado. Los clientes vuelven a la sala de espera sin reconectarse y el ciclo puede repetirse desde el paso 2.

3.2 Sistema de coordenadas

- Origen (0,0) ubicado en la esquina superior izquierda.
- El eje X crece hacia la derecha.
- El eje Y crece hacia abajo (Y negativo es arriba, Y positivo es abajo).
- El centro del mapa y del círculo es siempre (500, 500) (= `map_size / 2`).
- Todas las distancias se calculan con la distancia euclidiana estándar: $\text{dist}(a, b) = \sqrt{(a.x - b.x)^2 + (a.y - b.y)^2}$.

3.3 Reglas del dominio (normativas)

- **Spawn inicial:** Al enviar `start`, el servidor asigna a cada jugador una posición aleatoria uniforme en el anillo fuera del círculo: genera un ángulo $\theta \in [0, 2\pi)$ y un radio $R \in [350, 450]$, y calcula $x = 500 + R \cdot \cos(\theta)$, $y = 500 + R \cdot \sin(\theta)$. Esto garantiza apariciones equitativas y siempre fuera del círculo (el radio mínimo 350 es mayor que la distancia de victoria 315). Se permite que dos jugadores aparezcan traslapados: **no existe colisión entre cuerpos de jugadores**; la única interacción entre jugadores es el robo mediante `interact`.
- **Bandera libre:** Al inicio de la partida, y cada vez que el portador se desconecta, la bandera queda en el centro: `flag.owner = null`, `flag.x = 500`, `flag.y = 500`.
- **Captura:** Un `interact` de un jugador a distancia $\leq \text{interact_radius}$ (40) de la bandera libre lo convierte en portador.
- **Robo:** La distancia es el **único** requisito. Cualquier jugador a distancia ≤ 40 del portador

puede robarle la bandera con `interact`, sin importar si alguno de los dos está dentro o fuera del círculo. Sin tiempo de espera ni inmunidad: el robo es instantáneo.

- **Bandera portada:** Mientras haya portador, la bandera se transmite en la posición exacta del portador (`flag.x/flag.y = posición del jugador`).
- **Victoria (con transición obligatoria):** Un jugador gana cuando, siendo portador, pasa de estar **dentro o en el borde** del círculo (distancia al centro ≤ 315) a estar **completamente fuera** (distancia al centro > 315 , donde $315 = \text{circle_radius} + \text{player_radius}$). La máquina de estados del servidor registra, al momento de cada captura o robo, si el nuevo portador está dentro o fuera; quien roba estando ya fuera NO gana al instante: debe primero reingresar al círculo (distancia ≤ 315) y volver a salir completamente mientras conserva la bandera. Tocar el borde no basta: se exige distancia estrictamente mayor que 315.
- **Límites del mapa:** La posición de cada jugador se ajusta (clamp) al rango [15, 985] en ambos ejes ($= \text{player_radius}$ y $\text{map_size} - \text{player_radius}$).
- **Movimiento:** El servidor integra la última dirección `dir` recibida de cada jugador a `speed` (200) unidades por segundo en su propio paso de simulación; las diagonales se normalizan ($\div \sqrt{2}$) para que la rapidez sea idéntica en las 8 direcciones.

4. AUTORIDAD Y SINCRONIZACIÓN

4.1 Autoridad y validaciones

Acción	Qué envía el cliente	Qué valida el servidor	Resultado
Movimiento	Dirección dir.x y dir.y	Que los valores sean -1, 0, 1; que la partida esté activa y el jugador conectado.	Calcula la nueva posición.
Movimiento diagonal	Dirección en ambos ejes	Normaliza la dirección para evitar mayor velocidad.	Mantiene la misma rapidez en todas las direcciones.
Límites del mapa	No envía información adicional	Comprueba que la posición permanezca entre 15 y 985 en ambos ejes.	Ajusta la posición si intenta salir del mapa.
Captura de bandera	Mensaje interact	Que la bandera esté libre y que la distancia a la bandera sea menor o igual a 40.	Asigna la bandera al jugador.
Robo de bandera	Mensaje interact	Que otro jugador posea la bandera y que la distancia al portador sea menor o igual a 40. Es el único requisito: aplica dentro o fuera del círculo (sección 3.3).	Cambia el propietario de la bandera.
Condición de victoria	No envía un mensaje especial	Que el portador haya pasado de distancia ≤ 315 a distancia > 315 del centro conservando la bandera (sección 3.3).	Finaliza la partida y declara al ganador.
Mensaje recibido	Mensaje JSON	Que tenga type, los campos requeridos y los tipos de datos correctos.	Procesa el mensaje o responde con un error.
Fase de la partida	Acción correspondiente	Que la acción esté permitida en la fase actual.	Acepta o rechaza la acción.

Notas de implementación del servidor:

- **Cadencia de input:** El cliente envía **input** cada vez que su dirección cambia (incluido

el $(0,0)$ al soltar las teclas). No es obligatorio reenviarlo periódicamente; el servidor conserva y aplica la última dirección recibida hasta que llegue otra.

- **Evaluación dentro del tick:** En cada tick el servidor aplica primero el movimiento y la condición de victoria, y después procesa las interacciones pendientes. Un portador que cruza el límite en ese tick gana antes de que se evalúe cualquier robo pendiente.
- **Orden oficial:** Los mensajes se procesan de a uno en orden de llegada TCP; ese es el único criterio de desempate (sección 5.3).

4.2 Sincronización del estado

Aspecto	Decisión
Autoridad principal	El servidor
Posiciones oficiales	Las calcula el servidor
Información enviada por el cliente	Intenciones de movimiento e interacción
Captura, robo y victoria	Los valida el servidor
Límites y velocidad	Los controla el servidor
Función del cliente	Enviar acciones y mostrar el estado recibido
Mensajes state	Coalescibles: ante un cliente lento se descartan los pendientes y se envía solo el más reciente
Demás mensajes	Nunca se descartan ni se reordenan

5. MANEJO DE FALLAS Y DESCONEXIONES

5.1 Manejo de errores

Regla general: Cuando el servidor reciba un mensaje incorrecto, deberá:

1. Detectar el problema.
2. No modificar el estado del juego.
3. Responder con un mensaje error.
4. Mantener o cerrar la conexión según la gravedad.

5.1.1 Tabla de errores comunes

El campo **reason** del mensaje **error** DEBE ser exactamente uno de estos códigos (en MAYÚSCULAS_CON_GUIONES_BAJOS). No se inventan códigos nuevos sin actualizar este estándar.

Código / Error	Cuándo ocurre	¿Se cierra la conexión?
INVALID_JSON	El texto recibido no es un JSON válido.	No, excepto si ocurre repetidamente.
UNKNOWN_TYPE	El campo type contiene un mensaje desconocido.	No.
MISSING_FIELD	Falta un campo obligatorio.	No.
INVALID_FIELD	Un campo tiene un valor o tipo incorrecto.	No.
INVALID_PHASE	La acción no está permitida en la fase actual.	No.
VERSION_MISMATCH	Cliente y servidor usan versiones incompatibles.	Sí.
LOBBY_FULL	El servidor alcanzó el máximo de jugadores.	Sí.
NAME_INVALID	El nombre está vacío, es muy largo o no es válido.	No.
GAME_STARTED	Se recibió join con el servidor en countdown o playing.	Sí, después de enviar el error.
MESSAGE_TOO_LARGE	El mensaje supera el tamaño máximo permitido.	Sí.
NOT_JOINED	El cliente intenta jugar antes de enviar join.	No.

5.2 Desconexiones

Situación	Acción del servidor	Estado de la bandera
Desconexión en lobby	Eliminar jugador y actualizar lobby.	Sin cambios.
Desconexión en countdown	Eliminar jugador y comprobar <code>min_players</code> (2): si quedan menos de 2 jugadores, se aborta el countdown de inmediato y se envía en broadcast un mensaje lobby (regreso a la sala de espera).	Sin cambios.
Desconexión durante el juego	Eliminar jugador y continuar partida.	Sin cambios si no la tenía.
Desconexión del portador	Eliminar jugador.	Regresa a (500,500).
Se desconectan todos	Reiniciar la partida y volver al lobby.	Regresa a (500,500).
Desconexión en la pausa post-partida	Eliminar jugador; el lobby de regreso se envía con la lista ya actualizada.	Regresa a (500,500) al reiniciarse el ciclo.
Se desconecta el servidor	Los clientes detienen la partida.	No existe estado oficial.

5.2.1 Casos especiales de desconexión

- **TCP close explícito:** Eliminar sesión. En `playing`, el jugador se retira inmediatamente del `state`.
- **Sin timeout en v1:** La versión 1 del protocolo NO define timeout de inactividad. Un cliente que no envía nada (jugador quieto, o cliente en lobby/countdown) nunca es desconectado por el servidor. Las conexiones rotas se detectan por el propio TCP (cierre o error de escritura).
- **Portador se desconecta:** La bandera vuelve al centro: `flag.owner = null`, `flag.x = 500`, `flag.y = 500` en el siguiente `state`.
- **Servidor se desconecta:** Los clientes detectan el cierre de TCP, detienen la partida y muestran un aviso local de "servidor desconectado". No existe migración de servidor ni mensaje de protocolo para este caso.
- **Cliente intenta unirse a mitad de partida:** Se rechaza con `GAME_STARTED` durante `countdown` y `playing`, y la conexión se cierra. La reconexión a una partida en curso queda fuera de v1: volver a entrar solo es posible cuando el servidor está en `lobby`.

5.3 Decisiones de empate y propiedades del dominio

Caso	Regla autoritativa	Resultado observable
Dos capturan bandera libre	El servidor procesa los interact de a uno en orden de llegada TCP; el primero válido se queda con la bandera y el segundo se evalúa contra la bandera ya tomada (falla, salvo que esté a ≤ 40 del nuevo portador, en cuyo caso es un robo válido).	Un solo flag.owner en el siguiente state.
Varios roban al mismo portador	Mismo criterio: el primer interact válido en orden de llegada roba; los demás se evalúan contra el nuevo portador y deberán volver a intentarlo si no cumplen la distancia.	Un solo cambio de propietario por mensaje procesado.
Intento duplicado del mismo jugador	Un interact de quien ya es portador, o que no cumple ninguna condición, no tiene efecto.	El estado no cambia; no se envía error.
Cruce ganador y robo en mismo tick	Movimiento y victoria se evalúan antes que las interacciones (sección 4.1).	El portador que cruzó gana; no se procesa robo.
Frame parcial	El buffer espera los bytes restantes (sección 2.1).	No se considera mensaje hasta tener la línea completa.
Cliente lento	Los state pendientes se coalescen al más reciente; los demás mensajes se conservan (sección 4.2).	La memoria no crece; el cliente puede saltarse estados visuales.
Robo desde fuera del círculo	La distancia es el único requisito para robar, pero la victoria exige la transición dentro → fuera conservando la bandera (sección 3.3).	El ladrón que estaba fuera no gana al instante.

5.3.1 Propiedades que deben permanecer verdaderas

- En cualquier `state` existe como máximo un `flag.owner` no nulo.
- La bandera siempre tiene exactamente una ubicación: en el centro (500,500), en el punto donde quedó libre, o en la posición del portador.
- El ganador se escribe una sola vez por partida: un solo `game_over` por ronda.
- Cada mensaje recibido se procesa exactamente una vez y en orden de llegada; ningún

mensaje muta el estado dos veces.

- Un cliente no puede ganar enviando coordenadas ni declarando victoria: las posiciones y la victoria las calcula únicamente el servidor.
- La misma secuencia de mensajes, en el mismo orden de llegada, produce el mismo resultado de dominio (determinismo).

6. CONSIDERACIONES DE SEGURIDAD

6.1 Ausencia de Cifrado y Autenticación

El protocolo v1 transmite la información en texto plano mediante mensajes JSON sobre TCP/UDP. No implementa capas de cifrado (como TLS/DTLS) ni mecanismos de autenticación previa de usuarios (como tokens JWT o contraseñas).

- **Entorno de Red:** Se asume que el juego opera exclusivamente en una Red de Área Local (LAN) o entorno controlado de pruebas.
- **Riesgo:** Cualquier usuario conectado a la misma red puede interceptar el tráfico (sniffing) o enviar paquetes falsificados imitando el `player_id` de otro participante. Para este estándar, este riesgo es aceptado por el alcance del proyecto.

6.2 Denegación de Servicio (DoS) y Control de Memoria

Para evitar que un cliente malicioso o defectuoso agote los recursos de memoria o ancho de banda del servidor, la especificación impone tres barreras estrictas:

- **Límite de Tamaño de Mensaje (`message_max_size`):** Todo mensaje entrante que supere los 64 KB se rechaza con `MESSAGE_TOO_LARGE` y provoca el cierre inmediato del socket TCP. En UDP, los paquetes mayores a 64 KB se descartan silenciosamente.
- 1. **Saneamiento de Nombres (`name_max_length`):** Los nombres de jugadores están restringidos a un máximo de 20 caracteres UTF-8 tras aplicar recorte de espacios (trim) y eliminación de caracteres de control o saltos de línea.
- **Control de Conexiones (`max_players`):** El servidor debe limitar la sala a 100 conexiones simultáneas. Las solicitudes `join` excedentes se rechazan con `LOBBY_FULL` y el socket se cierra.

6.3 Validación de Entradas e Inyección en el Parser JSON

El servidor DEBE tratar todos los datos procedentes de los clientes como **no confiables**:

- **Inyección de Tipos (Type Confusion):** El parser debe verificar que las coordenadas o vectores de dirección (`dir.x`, `dir.y`) sean estrictamente tipos numéricos enteros y no cadenas de texto o estructuras complejas.
- **Campos Faltantes o Desconocidos:** Un mensaje con campos faltantes se responde con `MISSING_FIELD` sin modificar el estado. Los campos no documentados deben ignorarse silenciosamente para permitir extensibilidad sin desestabilizar el servidor.

6.4 Integridad de la Partida y Prevención de Trampas

Dado que el modelo es **100% Servidor-Autoritativo**, el cliente no tiene capacidad de dictar su posición global ni declarar victorias:

- **Restricción de Velocidad y Teletransporte:** El servidor no acepta coordenadas X,Y del cliente; solo acepta vectores de intención `dir` restringidos estrictamente a los valores `{-1, 0, 1}`. Si un cliente envía valores fuera de este rango, el servidor responde

INVALID_FIELD.

- **Transición de Victoria Obligatoria:** Para evitar que un cliente simule haber ganado sin cumplir el recorrido, el servidor valida internamente que el portador haya realizado la transición física de estar dentro del círculo (distancia ≤ 315) a estar completamente fuera (distancia > 315) conservando la bandera en todo momento.

7. CONTROL DE CAMBIOS

7.1 Versionado

- Este documento usa versionado semántico: **MAYOR.MENOR.PARCHE**.
- **MAYOR**: Cambios incompatibles en el cable (mensajes nuevos, campos eliminados o renombrados, reglas que cambian el comportamiento de otro proyecto). Un cambio MAYOR incrementa también el campo **v** del protocolo.
- **MENOR**: Campos opcionales nuevos o aclaraciones compatibles (todo proyecto anterior sigue interoperando gracias a la regla de campos desconocidos, sección 2.2). No cambia **v**.
- **PARCHE**: Correcciones de redacción, ejemplos o formato sin cambio de comportamiento. No cambia **v**.
- El campo **v** de los mensajes identifica la versión del **protocolo en el cable**, no la del documento; solo cambia con un cambio MAYOR.

7.2 Procedimiento para modificar el estándar

1. Cualquier estudiante propone el cambio al grupo completo (canal oficial de la clase), citando la sección afectada.
2. El cambio se discute y se aprueba por acuerdo de la clase; nadie modifica su implementación por cuenta propia.
3. Se actualiza este documento, se incrementa la versión y se registra en la tabla de cambios con fecha y autor.
4. Se notifica al grupo la nueva versión; todos los proyectos actualizan antes del siguiente día de pruebas.
5. El documento vive en un repositorio Git: cada versión aprobada queda como un commit identificable.

7.3 Historial de cambios

Versión	Fecha	Cambios	Autor(es)
1.0.0	2026-07-18	Versión inicial borrador. Contenía campos no definidos en el catálogo (flag_version, action_id, input_seq, mode, etc.), timeout de 8 s, códigos de error inconsistentes, flag.owner con valores ambiguos (null o 0), reglas de victoria y descubrimiento incompletas. Usado como base para la revisión grupal. No implementar.	Clase CC8 2026
1.1.0	2026-07-25	Versión consolidada. Cambios con respecto a 1.0.0: (1) eliminación de campos fantasma — todo desempate se basa en orden de llegada TCP + ejecución secuencial del servidor; (2) eliminación del timeout de 8 s (v1 no	Andrés Tobar

Versión	Fecha	Cambios	Autor(es)
		define timeout); (3) catálogo de errores unificado, con GAME_STARTED como nuevo código; (4) flag.owner libre fijado exclusivamente en null; (5) min_players=2 y aborto de countdown; (6) transición post-partida de 5 s con regreso al lobby; (7) rechazo de joins en countdown/playing; (8) desconexiones inferidas; (9) descubrimiento por broadcast dual; (10) SO_REUSEADDR/SO_REUSEPORT obligatorio; (11) centro del mapa fijado en (500,500) y victoria formalizada; (12) robo por distancia como único requisito; (13) bandera pegada al portador; (14) fórmula de spawn en anillo; (15) constantes de welcome.config declaradas fijas; (16) advertencia sobre \r\n; (17) límite de 64 KB; (18) lectura tolerante; (19) nueva sección 6. Incremento MENOR (1.1.0): el campo v se mantiene en 1.	
1.2.0	2026-07-25	Estructuración bajo formato RFC. Inclusión de Abstract, Terminología, Diagramas ASCII y Consideraciones de Seguridad.	Andrés Tobar